

# Инструкция и образец

## Производственная практика ПМ.08 — Разработка дизайна веб-приложений

7 семестр · Разработчик веб и мультимедийных приложений · Python / Flask / MySQL

09.02.07 — Информационные системы и программирование

ПМ.08 — Разработка дизайна веб-приложений

Учебные материалы СПО

### Конференции.РФ

Учебный разбор дизайна сервиса «Конференции.РФ» на Python, Flask и MySQL: запуск исходной основы, последовательное изменение интерфейса и проверка результата. Демонстрация не входит в 30 индивидуальных областей.



# Учебная задача: улучшить работающий интерфейс

## Результат разбора

Разработаем дизайн «Конференции.РФ»: пользователь выбирает площадку, регистрируется и отправляет заявку; администратор назначает и завершает мероприятие; пользователь оставляет отзыв. В этом руководстве используется только Python / Flask / MySQL. Решение показано в тексте и фрагментах кода; все необходимые действия объяснены здесь.

## Что изучаем в ПМ.08

Начальная основа содержит сервер, таблицы, каталог, формы и базовый интерфейс. Мы не переписываем весь сервер: улучшаем главное действие, обратную связь формы, доступность, карточки и мобильную компоновку. Каждый новый фрагмент добавляется в определённое место и заканчивается проверкой. Сначала выполните демонстрацию, затем создайте самостоятельный дизайн своей области.

## Последовательность

Запуск и исходный сценарий → проектирование страниц → подключение CSS/JS → кнопки и формы → карточки и мобильный экран → интерактивная подсказка → ошибки и фокус → полный сценарий → перенос в свою область. Сохраняйте рабочее состояние после каждого законченного изменения.

# Открыть начальную основу в редакторе

## Материалы для работы

[Скачать начальную основу Flask + MySQL \(ZIP\)](#)

Сервер и исходные страницы для учебного разбора. Дополнительный слой дизайна вы создаёте по коду методички.

## Проект для учебного разбора

Загрузите начальную основу ниже и распакуйте ZIP в отдельную папку. Это приложение до добавления учебных design.css и design.js; оно не является готовым индивидуальным проектом. В Visual Studio Code выберите File → Open Folder... / Файл → Открыть папку... и откройте projects/source/flask — папку с app.py. В левой панели должны быть видны app.py, templates, static, rooms.json и requirements.txt.

## Как устроены файлы

app.py принимает запросы и проверяет действия.  
mysql\_backend.py выполняет запросы к MySQL.  
templates/base.html задаёт общую оболочку, остальные шаблоны — содержимое страниц. static/css/style.css и static/js/app.js — исходное оформление и поведение.  
rooms.json содержит первоначальные площадки;  
schema-mysql.sql описывает таблицы. В этой практике мы добавляем два собственных файла и подключаем их в base.html.

## Проверить среду

Откройте Terminal → New Terminal / Терминал → Создать терминал. В Windows выполните py -3 --version, в macOS — python3 --version. Нужен Python 3.10 или новее и работающий локальный MySQL. Для просмотра таблиц используйте доступный MySQL Workbench или phpMyAdmin. Запишите фактические версии; не меняйте чужие базы.

# Создать отдельную базу MySQL

## Подготовка

Откройте SQL Editor в MySQL Workbench либо SQL в phpMyAdmin и подключитесь как администратор своего локального учебного MySQL. Выполните следующий фрагмент один раз. Если такая база или пользователь уже существуют, выберите новые имена и затем используйте их в командах запуска. Не удаляйте существующую базу ради повторного запуска.

## Учебный фрагмент: database.sql

```
-- Выполните под учётной записью администратора локального MySQL.  
CREATE DATABASE pm08_design_2027  
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;  
CREATE USER 'pm08_design'@'127.0.0.1'  
  IDENTIFIED BY 'Practice2027Db!';  
GRANT ALL PRIVILEGES ON pm08_design_2027.*  
  TO 'pm08_design'@'127.0.0.1';
```

## Почему отдельный пользователь

Учётная запись pm08\_design имеет доступ только к pm08\_design\_2027. Показанный пароль — учебное значение для локального стенда; параметры реальной среды задавайте в переменных окружения. Сервер MySQL должен быть запущен. Порт обычно 3306; если он иной, укажите его при запуске Flask.

## Как появятся таблицы

База пока пустая. При первом запуске Flask исходная основа применит schema-mysql.sql, загрузит шесть площадок из rooms.json и создаст учебного администратора. Поэтому не нужно вручную выполнять одновременно обе схемы или копировать таблицы из другого проекта. users хранит пользователей, rooms — площадки, applications — заявки, reviews — отзывы.

# Запустить Python и Flask в Windows

## Windows: PowerShell в VS Code

Выполните команды ниже в терминале папки с app.py. Виртуальное окружение отделяет зависимости этого проекта; прямой путь к python.exe позволяет работать без изменения ExecutionPolicy. pip устанавливает зависимости из приложенного requirements.txt. Если используете другую БД, пользователя или порт, замените соответствующие значения.

### Учебный фрагмент: run-windows.ps1

```
py -3 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
$env:DB_BACKEND="mysql"
$env:MYSQL_HOST="127.0.0.1"
$env:MYSQL_PORT="3306"
$env:MYSQL_DATABASE="pm08_design_2027"
$env:MYSQL_USER="pm08_design"
$env:MYSQL_PASSWORD="Practice2027Db!"
.\.venv\Scripts\python.exe -m flask --app app run --port 5017
```

## Проверка запуска

Откройте <http://127.0.0.1:5017/> . Должна появиться главная страница. Перейдите в «Помещения»: видно шесть площадок. В клиенте MySQL обновите список таблиц. Оставьте терминал сервера открытым; остановка выполняется Ctrl+C. Ошибку соединения ищите в фактических host/port/user/password, а не исправляйте изменением HTML.

# Запустить тот же проект в macOS

## macOS: тот же проект и стек

Для macOS выполните следующий набор команд вместо PowerShell-команд. Меняется синтаксис терминала, а Python, Flask, MySQL и весь учебный код остаются теми же.

## Учебный фрагмент: run-macos.sh

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
export DB_BACKEND=mysql
export MYSQL_HOST=127.0.0.1
export MYSQL_PORT=3306
export MYSQL_DATABASE=pm08_design_2027
export MYSQL_USER=pm08_design
export MYSQL_PASSWORD='Practice2027Db!'
.venv/bin/python -m flask --app app run --port 5017
```

## Проверка запуска

Откройте <http://127.0.0.1:5017/> . Должна появиться главная страница. Перейдите в «Помещения»: видно шесть площадок. В клиенте MySQL обновите список таблиц. Оставьте терминал сервера открытым; остановка выполняется Ctrl+C. Ошибку соединения ищите в фактических host/port/user/password, а не исправляйте изменением HTML.

# Проверить исходный пользовательский сценарий

## Обычный пользователь

Откройте регистрацию. Для учебной проверки: логин pm08student, пароль Practice2027, подтверждение того же пароля, ФИО «Практика Пользователь», телефон 8(900)123-45-67, почта student@example.test. Это вымышленные данные. Войдите, откройте каталог и выберите помещение. Создайте заявку «Обсуждение дизайна»: будущая дата, 12:00, 10 участников, способ оплаты СБП. После отправки она появляется в «Мои заявки» со статусом «Новая».

## Администратор и отзыв

Выйдите и войдите как Conf2027 / Demo77. Это учебная учётная запись администратора. Найдите заявку, выберите «Мероприятие назначено», сохраните; затем выберите «Завершено», сохраните. Выйдите и войдите как pm08student. Только после завершения доступны оценка и отзыв. Эти действия должны работать до изменения дизайна и после него.

## Граница сценария

Способ оплаты — выбранное значение, реального платежа нет. Приложение не предотвращает все пересечения бронирований. Оформление должно отражать реализованные возможности. Не обещайте оплату или автоматическую гарантию свободного зала.



# Спроектировать страницы и главное действие

## Каркас до CSS

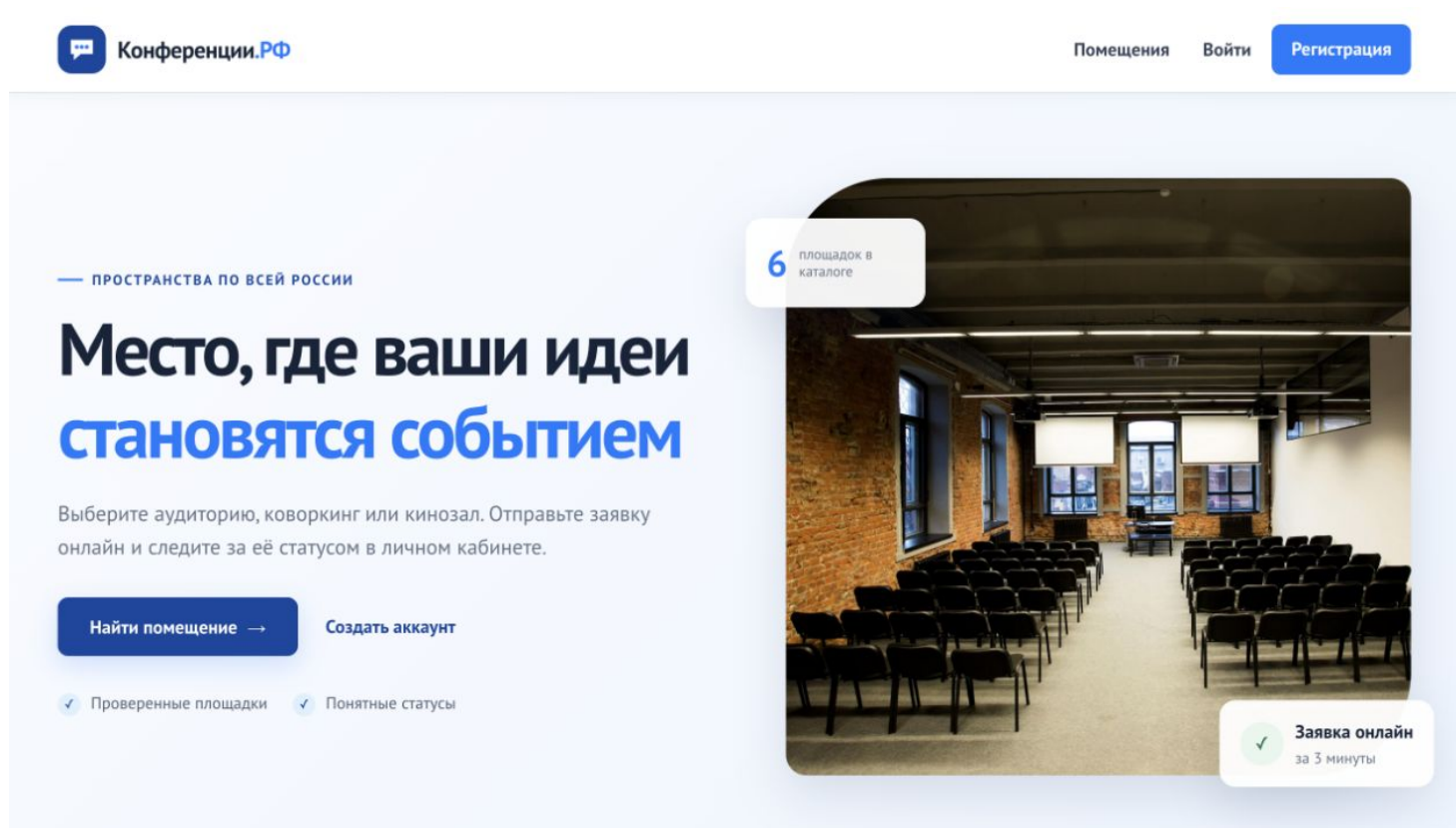
Нарисуйте простые прямоугольные каркасы: главная — объяснение сервиса и переход в каталог; каталог — фильтры и карточки; регистрация/вход — заголовок, поля и отправка; заявка — выбранная площадка и параметры; кабинет — статус и действия; администратор — поиск и изменение статуса. Расположите элементы в порядке пользовательской задачи, затем сравните каркас с существующими страницами.

## Палитра и типографика

В демонстрации основной синий #0d47a1, текст тёмный, фон светлый, шрифт PT Sans уже подключён локально. Белая подпись на #0d47a1 имеет контраст около 8,6:1. Цвет ошибки #a81727 сопровождается текстом. Базовый интервал 8 px помогает согласовать расстояния. Отдельно обозначьте главное и вторичное действие: «Выбрать» и «Подробнее» не должны одинаково привлекать внимание.



## Главная страница учебного примера



Первый экран объясняет назначение сервиса; основная кнопка ведёт к выбору площадки.

# Подключить собственный слой оформления

## Создать файлы

В Explorer / Проводник правой кнопкой нажмите static → New File / Новый файл. Создайте design.css, затем design.js. Оба файла сначала пустые. Откройте templates/base.html. Добавьте только две строки подключения из фрагмента в указанные места; поясняющие комментарии переносить необязательно. Сохраните Ctrl+S (macOS Cmd+S).

## Учебный фрагмент: design-includes.html

```
<!-- templates/base.html: после подключения css/style.css, внутри <head> -->
<link rel="stylesheet" href="{{ url_for('static', filename='design.css') }}">
```

```
<!-- templates/base.html: после js/app.js, перед закрывающим </body> -->
<script src="{{ url_for('static', filename='design.js') }}" defer></script>
```

## Почему именно так

base.html используется всеми страницами, поэтому общий дизайн подключается один раз. url\_for строит правильный адрес статического файла. CSS расположен после исходного оформления: равные по специфичности правила ниже имеют приоритет. defer запускает JavaScript после разбора HTML. Не заменяйте исходные css/style.css и js/app.js: в них уже работают сетки, меню и маска телефона.

## Проверка подключения

Откройте инструменты разработчика F12 (macOS Cmd+Option+I), вкладку Network / Сеть, обновите страницу. design.css и design.js должны иметь статус 200. Пока файлы пустые, вид не меняется. 404 означает неверное имя или расположение; синтаксическая ошибка JavaScript появится в Console.

# Задать общие цвета и интервал

## Место и смысл изменения

Начните пустой `static/design.css` с блока `:root`. CSS-переменные хранят общие значения; сами по себе они ещё не перекрашивают элементы. Акцент нужен главному действию и фокусу, цвет ошибки — неверному полю, интервал — согласованным расстояниям.

## Добавьте в `static/design.css` после предыдущего блока

```
/* Дизайн ПМ.08: добавляйте блоки в указанном порядке. */
:root {
  --design-accent: #0d47a1;
  --design-text: #162b48;
  --design-error: #a81727;
  --design-space: 8px;
}
```

## Проверка результата

В Elements выберите `body` и в Computed найдите `--design-accent`. Значение `#0d47a1` подтверждает загрузку файла; заметное изменение появится после следующего блока.

# Выделить главное действие и клавиатурный фокус

## Место и смысл изменения

Добавьте блок после `:root`. `.button-primary` и `.nav-primary` обозначают основные действия. `:not(.link-button)` сохраняет текстовый вид кнопки выхода. `min-height: 44px` увеличивает область нажатия. `:focus-visible` показывает фокус при навигации клавиатурой.

## Добавьте в `static/design.css` после предыдущего блока

```
/* Основное действие отличается от текста и имеет контрастную подпись. */
.button-primary, .nav-primary, button[type="submit"]:not(.link-button) {
  background-color: var(--design-accent);
  color: #fff;
  min-height: 44px;
}
a, button, input, select, textarea { touch-action: manipulation; }
:focus-visible {
  outline: 3px solid var(--design-accent);
  outline-offset: 4px;
}
```

## Проверка результата

В каталоге «Выбрать» заметнее, чем «Подробнее». Пройдите элементы Tab и Shift+Tab: фокус различим. Enter активирует выбранную кнопку/ссылку. Откройте меню и убедитесь, что «Выйти» не превратилось в главное действие.

# Оформить подсказку, поля и ошибку

## Место и смысл изменения

Добавьте следующий блок. `.design-summary` — оформление будущей подсказки; пока JavaScript не создан, она не появится. `calc` использует общий интервал. `.field-error` оформляет уже имеющийся текст ошибки. `aria-invalid` добавим после серверной проверки. Радиокнопки оценки визуально скрыты, но доступны клавиатуре; фокус выделяет их видимую метку.

## Добавьте в `static/design.css` после предыдущего блока

```
/* Формы: подсказка и ошибка сопровождают поле, а не только цвет. */
.design-summary {
  margin: 0 0 calc(2 * var(--design-space));
  padding: 12px calc(2 * var(--design-space));
  border-left: 4px solid var(--design-accent);
  border-radius: 8px;
  background: #e9f0fa;
  color: var(--design-text);
  font-size: 14px;
}
.design-summary strong { display: block; margin-bottom: 4px; }
[aria-invalid="true"] { border: 2px solid var(--design-error); }
.field-error { color: var(--design-error); }
.rating-options input { width: 1px; height: 1px; clip-path: inset(50%); }
.rating-options input:focus-visible + label { outline: 3px solid var(--design-accent); }
input, select, textarea { min-height: 44px; }
```

## Проверка результата

Откройте регистрацию и отправьте пустую форму. Прочитайте текст ошибок. Поля остаются доступными, метки и имена не меняются. После добавления JavaScript вернитесь к этой проверке: появятся рамка ошибочного поля и подсказка заполнения.

# Удержать карточку и длинное название в сетке

## Место и смысл изменения

Продолжите design.css блоком карточек. overflow-wrap позволяет переносить длинное название внутри карточки. aspect-ratio резервирует место изображения, object-fit: cover заполняет его с возможным обрезанием краёв. align-items: stretch выравнивает высоту карточек в строке.

## Добавьте в static/design.css после предыдущего блока

```
/* Карточки сохраняют форму при длинных названиях. */
.room-card h3, .application-card h3 {
  overflow-wrap: anywhere;
}
.room-card-image img {
  object-fit: cover;
  aspect-ratio: 16 / 10;
}
.room-grid { align-items: stretch; }
```

## Проверка результата

Откройте каталог на широком экране. Картинки имеют одинаковое соотношение сторон. Проверьте, что смысловой объект виден после обрезания. В своём каталоге испытайте длинное название: оно не должно выходить в соседнюю колонку.

# Перестроить интерфейс для мобильного экрана

## Место и смысл изменения

Добавьте media query: правила действуют до 600 px. Каталог становится одноколоночным, отступы формы уменьшаются. minmax(0, 1fr) допускает сжатие колонки. Мы меняем компоновку, а не маскируем ошибку через overflow-x: hidden.

## Добавьте в static/design.css после предыдущего блока

```
/* Узкий экран: ничего не скрываем ради устранения переполнения. */
@media (max-width: 600px) {
  .form-card { padding: 20px; }
  .room-grid { grid-template-columns: minmax(0, 1fr); }
  .design-summary { padding: 12px; }
  .filter-panel input, .filter-panel select { max-width: 100%; }
  h1 { overflow-wrap: anywhere; }
}
```

## Проверка результата

В DevTools включите Device toolbar / Панель устройств, Responsive, ширину 390 и высоту 844. Просмотрите каждую страницу, включая вход, заявку, кабинет и администратора. Затем верните широкий экран: каталог снова многоколоночный.

# Добавить короткую обратную связь без лишнего движения

## Место и смысл изменения

Завершите `design.css`. Подсказка появляется с небольшой анимацией. Системное предпочтение `prefers-reduced-motion: reduce` отключает движение и переходы. Это улучшение не меняет отправку формы и не откладывает доступность полей.

## Добавьте в `static/design.css` после предыдущего блока

```
/* Анимация обратной связи не мешает читать и вводить данные. */
.design-summary { animation: design-appear .18s ease-out; }
@keyframes design-appear {
  from { opacity: 0; transform: translateY(4px); }
  to { opacity: 1; transform: translateY(0); }
}
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation: none !important;
    transition: none !important;
    scroll-behavior: auto !important;
  }
}
```

## Проверка результата

После подключения JavaScript обновите регистрацию и увидите появление подсказки. В DevTools → More tools → Rendering включите `prefers-reduced-motion: reduce`. Подсказка остаётся видимой и работоспособной, но движение исчезает.



# Понять связь формы, Flask и сообщения ошибки

## Разбор существующей регистрации

Откройте `templates/register.html` и найдите группу логина внутри `.stack-form`. Ниже приведён её фрагмент с переносами для чтения; он уже есть в основе, повторно вставлять его не нужно. `for/id` связывают подпись с полем, `name="username"` соответствует ключу `request.form` в Python, `values.username` сохраняет исправляемый ввод. `errors` приходит от Flask после POST. Выражения `{{ ... }}` подставляют данные, `{% if ... %}` выводит сообщение только при ошибке.

## Учебный фрагмент: `username-field.html`

```
<div class="field {% if errors.get('username') %}has-error{% endif %}">
  <label for="username">Логин <span>*</span></label>
  <input id="username" name="username" value="{{ values.username }}"
    required minlength="6" pattern="[A-Za-z0-9]+"
    autocomplete="username"
    aria-describedby="username-hint{% if errors.get('username') %} username-error{% endif %}">
  <small id="username-hint">Латиница и цифры, не менее 6 символов</small>
  {% if errors.get('username') %}
    <span class="field-error" id="username-error">{{ errors.username }}</span>
  {% endif %}
</div>
```

## Почему сервер проверяет ввод

`required` и `pattern` помогают описать поле, но не заменяют серверную проверку. Исходная форма имеет `novalidate`, чтобы продемонстрировать единые сообщения Flask. Скрытый `_csrf_token` связывает отправку с сессией. Не меняйте `name`, `method="post"` и CSRF ради внешнего вида. Ошибка должна быть понятна по тексту, а `aria-describedby` связывает подсказку и ошибку с конкретным полем.

# Показать заполнение обязательных полей

## Что пишем в JavaScript

В пустой `static/design.js` введите первый блок. `.stack-form` выбирает основную форму, а `required` отделяет обязательные поля. `hidden` исключает CSRF. У радиокнопок одного `name` общий ответ: считаем группу один раз. Создаём текстовые элементы безопасно через `textContent`, помещаем подсказку перед полями и обновляем её на `input/change`.

## Первый блок `static/design.js`

```
// ПМ.08: подсказка сопровождает основную форму, сервер проверяет данные.
'use strict';
const designForm = document.querySelector('.stack-form');
if (designForm) {
  const allFields = [...designForm.querySelectorAll('input, select, textarea')];
  const fields = allFields.filter((field, index) => field.type !== 'hidden' && field.required
    && (field.type !== 'radio' || allFields.findIndex(other => other.name === field.name) === index));
  const summary = document.createElement('p');
  summary.className = 'design-summary';
  summary.setAttribute('role', 'status');
  const heading = document.createElement('strong');
  heading.textContent = 'Заполнение формы';
  const counter = document.createElement('span');
  summary.append(heading, counter);
  designForm.prepend(summary);
  function updateSummary() {
    const filled = fields.filter(field => field.type === 'radio'
      ? allFields.some(other => other.name === field.name && other.checked)
      : field.value.trim() !== '').length;
    counter.textContent = `Заполнено ${filled} из ${fields.length} обязательных полей. Проверка выполняется при отправке.`;
  }
  designForm.addEventListener('input', updateSummary);
  designForm.addEventListener('change', updateSummary);
  updateSummary();
}
```



## Как читать updateSummary

Обычное поле заполнено, если его обрезанное значение не пустое. Группа radio заполнена, когда хотя бы одна кнопка отмечена. Счётчик показывает факт заполнения, а не правильность телефона, пароля или даты. role="status" обозначает обновляющееся сообщение. Начальный вызов updateSummary показывает состояние сразу, а if защищает страницы без .stack-form.

## Проверка по действиям

На регистрации: 0 из 6 → введите логин → 1 из 6 → очистите логин → 0 из 6. На новой заявке выбор одного способа оплаты увеличивает счётчик на один; переключение способа не увеличивает число обязательных полей. Отправьте корректную форму: серверный сценарий сохраняется. Откройте каталог: в Console нет ошибки обращения к отсутствующей форме.

# Связать серверную ошибку с полем и поставить фокус

## Дополнение к подсказке

Добавьте второй блок в конец `static/design.js`, после закрывающей скобки первого `if`. На уже отрисованной странице ищем `.field-error`, затем поле внутри ближайшей `.field`. `aria-invalid` помечает неверный ввод. `Set` сохраняет существующие подсказки и исключает повторение идентификаторов. Фокус получает первое ошибочное поле.

## Второй блок `static/design.js`

```
// Связываем видимые серверные ошибки с соответствующими полями.
const errors = document.querySelectorAll('.field-error');
let firstInvalid = null;
errors.forEach((error, index) => {
  if (!error.id) error.id = `design-error-${index}`;
  const group = error.closest('.field');
  // Поле и его ошибка находятся внутри одной группы .field.
  const input = group ? group.querySelector('input, select, textarea') : null;
  if (!input || !input.matches('input, select, textarea')) return;
  input.setAttribute('aria-invalid', 'true');
  const ids = new Set((input.getAttribute('aria-describedby') || '').split(/\s+/).filter(Boolean));
  ids.add(error.id);
  input.setAttribute('aria-describedby', [...ids].join(' '));
  firstInvalid ||= input;
});
if (firstInvalid) firstInvalid.focus();
```

## Проверка результата

Отправьте пустую регистрацию. Курсор должен быть в логине. В Elements у него видны `aria-invalid="true"` и `aria-describedby` с `username-hint` и `username-error`. Исправьте форму и отправьте повторно. Ошибки исчезают после успешной серверной проверки. Фокус не должен неожиданно перемещаться при обычном вводе: этот блок выполняется при загрузке ответа с ошибками.

## Регистрация: текст ошибок и фокус в логине

The screenshot shows a registration page for 'Конференции.РФ'. At the top, there are links for 'Помещения', 'Войти', and 'Регистрация'. The main heading is 'Создайте учётную запись' (Create an account) with a subtext 'Заполните все поля. После регистрации вы сможете бронировать помещения.' (Fill in all fields. After registration, you will be able to book rooms.).

A red error message box at the top of the form states: 'Проверьте отмеченные поля.' (Check the marked fields.). Below this, a blue box indicates 'Заполнение формы' (Form filling) with the text 'Заполнено 0 из 6 обязательных полей. Проверка выполняется при отправке.' (0 out of 6 required fields filled. Check is performed upon submission.).

The form fields are arranged in two columns:

- Логин \*** (Login \*): A text input field containing the letter 'I'. Below it, a red error message says 'Латиница и цифры, не менее 6 символов. Не менее 6 символов: только латиница и цифры.' (Latin letters and digits, at least 6 characters. At least 6 characters: only Latin letters and digits.).
- ФИО \*** (Full Name \*): A text input field containing 'Иванов Иван Иванович'.
- Телефон \*** (Phone \*): A text input field containing '8(900)123-45-67'. Below it, a red error message says 'Формат телефона: 8(000)XXX-XX-XX.' (Phone format: 8(000)XXX-XX-XX.).
- Электронная почта \*** (Email \*): A text input field containing 'name@example.ru'. Below it, a red error message says 'Укажите ФИО кириллицей, между словами – пробелы. Введите корректный адрес электронной почты.' (Specify the full name in Cyrillic, with spaces between words. Enter a correct email address.).
- Пароль \*** (Password \*): A password input field with a visibility toggle icon. Below it, a red error message says 'Не менее 8 символов. Пароль должен содержать от 8 до 128 символов.' (At least 8 characters. Password must contain 8 to 128 characters.).
- Повторите пароль \*** (Repeat password \*): A password input field with a visibility toggle icon.

At the bottom of the form is a blue button labeled 'Создать пользователя' (Create user) with a right arrow. Below the button, a link says 'Уже зарегистрированы? Войти' (Already registered? Log in).

The footer contains the company logo 'Конференции.РФ', a description 'Помогаем находить пространства для событий по всей России.', a navigation link 'Каталог помещений', contact information '8 (800) 555-20-27, hello@conference.test', and a copyright notice '© 2022 Конференции.РФ - Учебный проект'.

Ошибка выводится сервером рядом с полем; CSS делает её заметной, JavaScript связывает сообщение и переводит фокус.



# Проверить каталог, фильтр и пустой результат

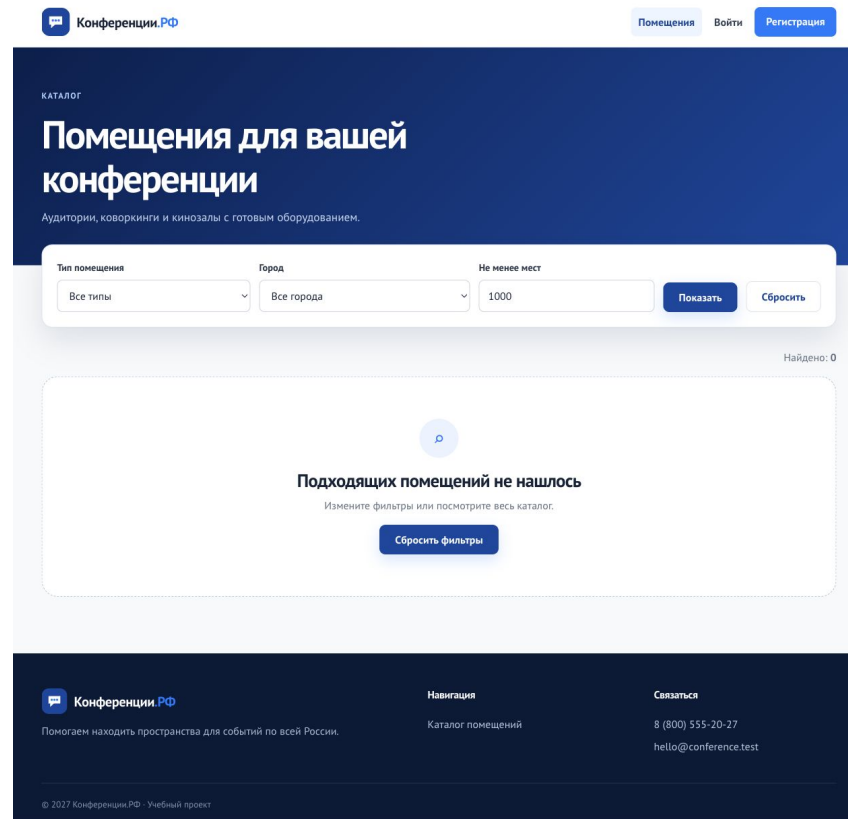
## Как данные становятся карточкой

Flask читает rooms из MySQL и передаёт список в templates/rooms.html. Для каждой записи подключается \_room\_card.html. Поля name, description, city, capacity, hourly\_rate и image определяют содержимое; CSS определяет вид. Фильтр отправляет GET-параметры и сохраняет выбранные значения. Это позволяет видеть условие текущего результата.

## Проверка состояний

Выберите тип помещения и нажмите «Показать»: остаются соответствующие карточки, выбранный тип сохраняется. Укажите минимальную вместимость 1000: список пуст, появляется сообщение и возможность сбросить фильтр. Верните все помещения. Пустой результат отличается от ошибки соединения с MySQL — у них разные причины и сообщения.

## Каталог: пустой результат фильтра



При вместимости 1000 площадок не найдено. Условия сохраняются, сообщение объясняет результат и предлагает сбросить фильтр.



# Проверить мобильную регистрацию и навигацию

## Один сценарий на двух размерах

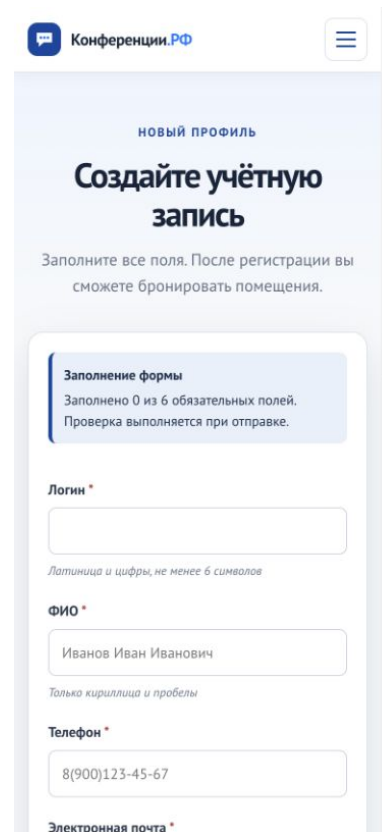
При 390×844 поля регистрации стоят последовательно, основная кнопка доступна после прокрутки, меню раскрывается отдельной кнопкой. На широком экране форма снова использует колонки. Откройте меню клавиатурой, пройдите ссылки и закройте его. Проверьте также личный кабинет под обычным пользователем и панель под администратором: это разные экраны.

## Если появился горизонтальный скролл

В Elements найдите элемент, который шире своей колонки. Проверьте фиксированную ширину, min-width, длинный непрерывный текст и grid-колонки. Исправьте причину. Вертикальная прокрутка длинной формы ожидаема; горизонтальный выход поля за экран мешает вводу.



## Регистрация при 390×844



Конференции.РФ

новый профиль

### Создайте учётную запись

Заполните все поля. После регистрации вы сможете бронировать помещения.

**Заполнение формы**

Заполнено 0 из 6 обязательных полей.  
Проверка выполняется при отправке.

**Логин \***

Латиница и цифры, не менее 6 символов

**ФИО \***

Только кириллица и пробелы

**Телефон \***

**Электронная почта \***

Мобильный снимок работающей регистрации: заголовок, подсказка и поля помещаются в ширину экрана.



# Повторить заявку, статусы и отзыв после изменения дизайна

## Проверка сохранности сценария

Войдите как обычный пользователь. Создайте новую заявку и откройте кабинет. Выйдите, войдите как администратор, назначьте мероприятие и завершите его. Снова войдите как пользователь и оставьте отзыв. Не проверяйте пользовательскую форму под администратором: приложение направляет его в панель управления.

## Как показывать состояние

Статус должен иметь видимый текст: «Новая», «Мероприятие назначено», «Завершено». Цвет помогает различить состояния, но не заменяет подпись. После сохранения действия показывается сообщение результата. Отзыв доступен после завершения; CSS не должен делать недоступное действие похожим на рабочее.

## Панель администратора и сохранённая заявка

The screenshot displays the 'Панель администратора' (Administrator Panel) of the 'Конференции.РФ' (Conferences.RF) system. The interface includes a top navigation bar with links for 'Помещения' (Rooms), 'Панель администратора' (Administrator Panel), 'Conf2027', and 'Выйти' (Logout). A status message at the top right indicates: 'Статус заявки №1: Мероприятие назначено.' (Request #1 status: Event assigned).

The main section is titled 'Панель администратора' with the subtitle 'Проверяйте заявки и обновляйте их статусы.' (Check requests and update their status). It features a summary row with four cards: 'Всего заявок' (Total requests) with a count of 1, 'Новые' (New) with 0, 'Назначены' (Assigned) with 1, and 'Завершены' (Completed) with 0.

Below this is a search and filter section with the following controls:

- Поиск** (Search): 'Заявка, клиент или площадка' (Request, client or venue)
- Статус** (Status): 'Все статусы' (All statuses)
- Тип** (Type): 'Все типы' (All types)
- Оплата** (Payment): 'Любая' (Any)
- Сортировка** (Sorting): 'Сначала новые' (Newest first)

A 'Применить' (Apply) button is located to the right of the filters.

The 'Заявки' (Requests) section shows a list of requests. The first request is highlighted with a 'Мероприятие назначено' (Event assigned) status. The request details are: 'Заявка №1 · 30.09.2026' and 'Обсуждение дизайна' (Design discussion). An 'Изменить статус' (Change status) button is visible next to the request.

Администратор видит заявку и управляет её статусом. Учебные данные в снимке вымышленные.

# Перенести учебные приёмы в свою предметную область

## Новая БД и свой каталог

Найдите свой номер в «Списке предметных областей» и загрузите его JSON. В новой копии исходной основы замените rooms.json до первого запуска. Создайте отдельную пустую БД с новым именем и обновите MYSQL\_DATABASE; предоставьте своему пользователю права на неё. При первом запуске загрузятся три ресурса области. Замена JSON в уже заполненной БД не обновляет автоматически строки MySQL.

## Что должно стать самостоятельным

Измените названия сервиса, ресурсов, событий и действия по смыслу области. Технические значения auditorium/coworking/cinema сохраняйте для фильтра, а видимые подписи задавайте через ROOM\_TYPE\_LABELS в app.py. Разработайте палитру, типографику и композицию под своего пользователя. Изображения должны соответствовать ресурсу; примерные адреса и цены в каталоге вымышленные.

## Два дополнительных изменения

Прочитайте оба условия своей области. Для каждого запишите пользовательскую проблему, место изменения, решение и фактическую проверку. Например, оснащение помогает понять, подходит ли зал; перенос длинного названия позволяет прочесть событие на мобильном экране. Простая замена цвета сама по себе не объясняет решение пользовательской задачи.

# Пример небольшого изменения: показать оснащение

## Проблема и место

Пользователь видит вместимость, но не знает, есть ли нужное оборудование. В поле equipment каталога эти данные уже есть. В templates/\_room\_card.html найдите `<p>{{ room['description'] }}</p>` и добавьте после него следующий фрагмент. Это дополнительный учебный приём; применяйте его там, где этого требует ваша область.

## Учебный фрагмент: equipment.html

```
<!-- templates/_room_card.html: после <p>{{ room['description'] }}</p> -->
{% if room['equipment'] %}
  <p class="room-equipment"><strong>Оснащение:</strong> {{ room['equipment'] }}</p>
{% endif %}
```

## Как устроено решение

if не выводит пустую строку при отсутствии оборудования. Jinja экранирует значение, поэтому обычный текст показывается как текст. Не используйте |safe для данных каталога. В static/design.css своей области оформите .room-equipment как вторичную информацию, сохраняя читабельность. Не подменяйте основное действие декоративным блоком.

## Проверка

Откройте карточку с заполненным equipment: текст появился после описания. Испытайте длинную строку на 390×844 и отсутствие equipment. Повторите выбор площадки и создание заявки: добавление HTML не должно менять параметры отправки.

# Найти и исправить типичные ошибки

## CSS не применяется

В Network проверьте статус 200 и содержимое design.css. В Elements → Styles найдите своё правило: зачёркнутое правило уступило более специфичному или расположенному ниже. Проверьте имя класса и порядок подключений. Не увеличивайте специфичность наугад и не добавляйте !important ко всему файлу.

## JavaScript остановился

Console показывает строку первой синтаксической ошибки. Проверьте скобки, кавычки и имя файла. На странице без формы designForm равен null — это штатно, код внутри if не выполняется. Подсказка не должна вмешиваться в method, action, name и скрытый CSRF.

## Каталог или форма не работают

Если каталог не обновился, проверьте выбранную новую БД и первый запуск с нужным rooms.json. Если ответ 400, сначала проверьте сохранность CSRF и обновите форму. Если ответ 503, проверьте доступность MySQL. Не маскируйте серверную ошибку стилями или постоянным сообщением «успешно».

# Подтвердить результат и объяснить решения

## Функциональная проверка

Пройдите регистрацию с ошибкой и успехом, вход, фильтр и пустой результат, заявку, чужую заявку, смену статусов и отзыв. В терминале проекта для тестов задайте `PORTAL_TEST_MYSQL=1` и `MYSQL_PORT` своего сервера; выполните `python окружения -m pytest -q`. Тесты создают отдельную временную БД `rvpm_test_...`; выбранному тестовому пользователю нужны права её создания. Не выдавайте приложению глобальные права ради проверки — используйте отдельную тестовую учётную запись администратора. Ожидается 8 passed.

## Визуальная и пользовательская проверка

Просмотрите каждую страницу на широком экране и 390×844. Проверьте основное действие, вторичные кнопки, длинные названия, ошибку рядом с полем, клавиатурный фокус и `reduced motion`. Сопоставьте результат с каркасом: объясните, как каждое изменение помогает пользователю выполнить действие.

## Git и отчёт

Зафиксируйте осмысленные изменения в Git. Не включайте виртуальное окружение, сессии и реальные секреты. Заполняйте общий шаблон отчёта своими решениями и фактическими результатами проверки. При сдаче проекта приложите ссылку на его репозиторий. Снимки должны показывать ваш интерфейс, а не повторять учебную демонстрацию.